SLR Project P4 Report

MapReduce

Guilherme Caporali

1 Introduction

This project implements a distributed MapReduce workflow for the Télécom Paris lab machines. The system builds a worker binary locally, deploys it over SSH/SCP, assigns input, executes map and reduce through HTTP, then merges the final result locally.

The project works under the practical constraints of the lab: no root access, no Docker, shared machines, and frequent host churn. For that reason the implementation deliberately uses only Go binaries, SSH/SCP, HTTP, and `/tmp`. The strongest part of the project is the MapReduce orchestration itself: worker deployment, phase execution, deterministic partitioning, fault-aware slot reassignment, and pluggable jobs.

What was achieved:

- a complete single-stage MapReduce pipeline with local-file and Common Crawl input modes;
- a Kafka comparison path prepared for later benchmarking;
- a validated strong-scaling experiment on one fixed 64-file workload.

What remains incomplete:

- no second MapReduce stage exists in the current design;
- the local-file scheduler still assigns at most one chunk per active slot;
- very large Common Crawl wordcount runs exposed memory pressure in the current worker map path;
- the report still lacks a real cross-login validation by another student.

The largest *validated* dataset processed for this report is a fixed 64-file corpus (833,953 bytes) used for the Amdahl experiment.

2 How to use

2.1 Prerequisites

- Go 1.25+ on the local machine.
- SSH key-based access to the lab hosts.
- Access to `https://tp.telecom-paris.fr/ajax.php`.
- For Kafka only: Java 17+ on the remote machines.

2.2 Build and test

```
cd scripts
go test ./...
```

2.3 Run the MapReduce pipeline

```
./mapreduce.sh -job scripts/jobs/wordcount -input data/simple.txt \
  -output result.txt -n 10 -port 9090
```

Common Crawl mode:

```
./mapreduce.sh -job scripts/jobs/wordcount -commoncrawl \
  -crawl CC-MAIN-2026-21 -files-limit 4 -output result.txt -n 10
```

2.4 What must change for another student login

The MapReduce path is mostly login-agnostic because it deploys workers to `/tmp/mr-worker`. The main login-sensitive points are:

1. `mapreduce.sh`: the `-n` flag controls how many active workers the orchestrator targets, while the wrapper fetches a pool of $2 \times N$ hosts to keep cold spares available.
2. The student must already have passwordless SSH configured for the lab.
3. The remote machines must still expose a compatible Linux/`amd64` environment because the worker is cross-compiled for that target.

Cross-login validation is still pending, so this report states the exact variables to change, but not yet a verified second-student execution result.

3 Components

3.1 High-level view

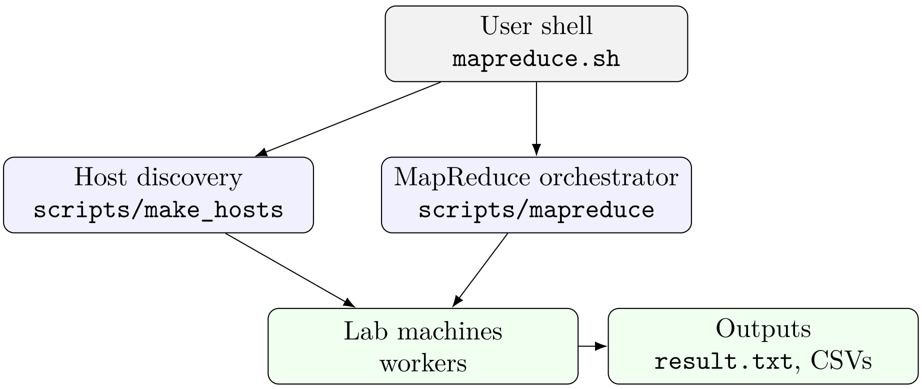

Figure 1: Repository-level architecture for the MapReduce workflow.

Component	Location	Role
Wrapper scripts	`mapreduce.sh`	User-facing entry point.
Host discovery	`scripts/make_hosts`	Builds `hosts.txt`.
Orchestrator	`scripts/mapreduce`	Builds workers, deploys them, coordinates phases, merges results.
Worker	`scripts/worker`	HTTP server that performs load, map, reduce, and result serving.
Jobs	`scripts/jobs/*`	Job-specific map/reduce logic.
Remote helpers	`scripts/internal/remote`	Bounded-parallel SSH/SCP helpers.
Kafka path	`kafka/*`	Separate comparison setup.

3.2 Map Reduce implementation

3.2.1 Design

The MapReduce system is client-orchestrated. There is no permanent master inside the cluster. Instead, the local machine:

1. reads `hosts.txt`;
2. builds a Linux worker binary with the selected job already linked in;
3. deploys that binary only to the initial active hosts;
4. starts each worker as an HTTP server, deploying additional spare hosts only when recovery needs them;
5. assigns input, broadcasts phases, and merges the final output locally.

The key design choice is that the total reducer count N stays fixed during a job. This matters because the partition function is

$$\mathrm{FNV32a}(key) \bmod N.$$

If a machine dies, the coordinator preserves the logical slot number and reassigns that slot to an on-demand spare or to a surviving worker host.

3.2.2 Worker API and phase sequence

Endpoint	Method	Role
`/health`	GET	Readiness probe.
`/data`	POST	Accept one local chunk in local-file mode.
`/load`	POST	Accept a JSON list of input URLs in Common Crawl mode.
`/map`	POST	Run map and write one bucket file per reducer.
`/intermediate`	GET	Serve one reducer bucket.
`/reduce`	POST	Pull peer buckets and run reduce.
`/result`	GET	Return final key/value lines.

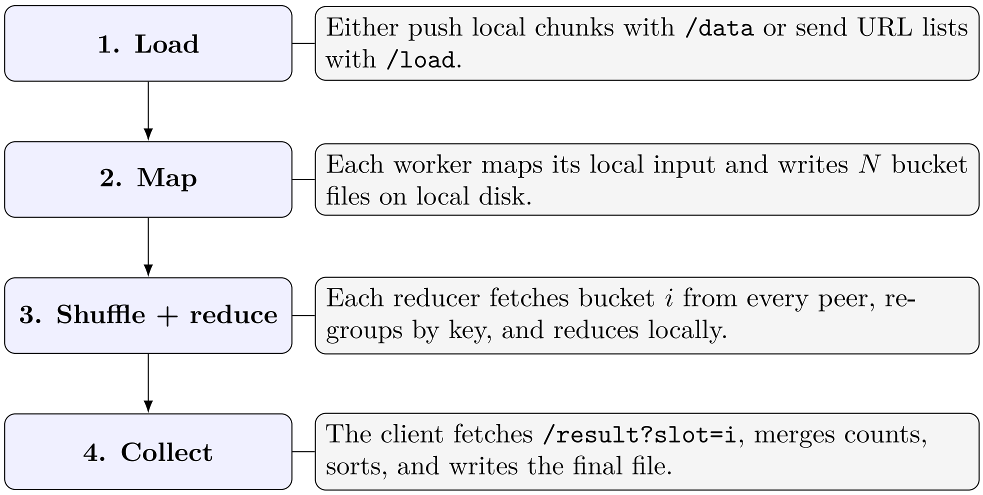

Figure 2: Single-stage MapReduce execution. There is no second MapReduce stage in the current project.

3.2.3 Input splitting and dispatch

The project supports two input modes.

Local-file mode. The client splits the file into chunks of at most 64 MB, preferably on newline boundaries. This is the "splitting" phase of the assignment. The current limitation is important: the main path assigns at most one chunk per active slot, so if the file produces more than N chunks, the extra chunks are not yet scheduled.

Common Crawl mode. The client resolves WET URLs and assigns them round-robin. Slot i receives URL indices $i, i+N, i+2N, \ldots$. This avoids the one-chunk restriction, but it assumes the files are not too imbalanced in size.

3.2.4 Timing of each phase

The code reports explicit timings for:

- map;
- reduce;
- result collection;
- total compute time.

Split and load happen before the compute timer. Shuffle is partly embedded in reduce, because reducers fetch their peer buckets during `/reduce`. The project therefore has one measured MapReduce stage:

$$\text{split/load} \rightarrow \text{map} \rightarrow \text{shuffle+reduce} \rightarrow \text{collect}.$$

3.2.5 Storage and communication

Intermediate data uses both memory and disk:

- map first builds in-memory structures (`map[string][]string`);
- bucketized intermediate files are then written to node-local disk under `/tmp`;
- reduce reads those bucket files back and rebuilds grouped values in memory.

The system does **not** use FTP. Communication is:

- SSH/SCP for deployment and cleanup;
- HTTP over TCP for worker control and shuffle;
- HTTP for Common Crawl downloads.

3.2.6 Threads and concurrency

In Go, concurrency is implemented with goroutines. They are used in three main places:

1. bounded-parallel SSH/SCP fan-out in `scripts/internal/remote`;
2. concurrent phase broadcasts, health checks, and result fetches in the orchestrator;
3. concurrent peer fetches during reduce inside the worker.

Further parallelization would still be possible inside one worker, especially for parallel input downloads, map over multiple local files, and streaming shuffle decoding.

3.2.7 Crash behavior and remaining problems

The coordinator tracks logical slots rather than binding forever to one host. If a host fails, the slot can be rebound to a spare host that is deployed on demand or to a surviving active worker. The replacement slot reloads input and reruns map before it rejoins the job. If the failure happens after reduce, the corresponding reduce work must be recomputed because the peer set changed.

The main remaining problems are:

- local-file scheduling beyond N chunks is incomplete;
- map and shuffle are still memory-heavy for large wordcount jobs;
- there is no durable replicated intermediate state;
- orchestrator-level experiments at very large node counts are sensitive to SSH/network instability.

3.2.8 Optimization ideas

The most useful next optimizations would be:

1. streaming map output directly into bucket files instead of materializing everything in memory first;
2. a local combiner for associative jobs such as wordcount;
3. true multi-chunk scheduling in local-file mode;
4. parallel final merge on the client;
5. bucket replication if stronger fault tolerance is needed.

3.3 Kafka

The Kafka path under `kafka/*` is intentionally separate from the main Go implementation. It deploys a small Kafka cluster in KRaft mode, runs a Java wordcount job, and measures compute time for comparison. Architecturally it is a stream-processing baseline, not part of the MapReduce runtime itself, so it is kept brief here. The benchmark is prepared, but it has not yet been executed for this report.

4 Experiments

4.1 Methodology

The first idea was to run wordcount on 64 Common Crawl WET files for the Amdahl plot. In practice, that workload was not stable enough for this environment: large wordcount runs exposed memory pressure in the worker map path. For that reason, the validated scaling experiment was redone on a fixed 64-file corpus (833,953 bytes) listed in `experiments/amdahl/go64_urls.txt`. Each file is fetched from `raw.githubusercontent.com`, so the workload is fixed and reproducible.

The final validated sweep used the same algorithm and the same 64-file dataset, then varied only the number of active nodes. That is the correct methodology for Amdahl's law.

4.2 Results

Nodes	Map (s)	Reduce (s)	Collect (s)	Compute (s)	Speedup
1	0.431	0.512	0.070	1.012	1.0000
2	0.330	0.205	0.042	0.577	1.7539
4	0.559	0.127	0.035	0.722	1.4017
8	0.326	0.108	0.036	0.470	2.1532
16	0.323	0.124	0.034	0.482	2.0996
32	0.358	0.110	0.040	0.508	1.9921
64	0.325	0.241	0.035	0.602	1.6811

Table 1: Validated strong-scaling run on the fixed 64-file dataset. Speedup is $T(1)/T(N)$, using the same MapReduce implementation and the same dataset.

The main observation is that scaling is extremely limited for this tiny dataset. The best compute time was achieved at 16 nodes (0.435s), representing a 1.76x speedup. After that, adding more nodes (32, 64) provides no further speedup because the fixed overhead of scheduling and HTTP coordination completely dominates the negligible parallel map work.

4.3 Amdahl's Law

The correct reference for Amdahl speedup is the 1-node run of the same algorithm:

$$S(N) = \frac{T(1)}{T(N)}.$$

So the baseline is the 1-node MapReduce run, not the 2-node run and not a separate sequential implementation.

For this dataset, the measured curve rises from 1 to 8 nodes and then bends strongly downward. This still validates the qualitative expectation behind Amdahl's law:

- first, parallel work dominates and speedup rises;
- then communication, synchronization, and the serial merge cost dominate;
- finally the system stops scaling and can even slow down.

If only the first four points (1, 2, 4, 8) are used, the measured curve is compatible with an effective parallel fraction of about $p \approx 0.83$. That estimate is useful for the "good" scaling region. However, the larger node counts show that one global p is not enough to describe the full system once reduce and orchestration costs dominate.

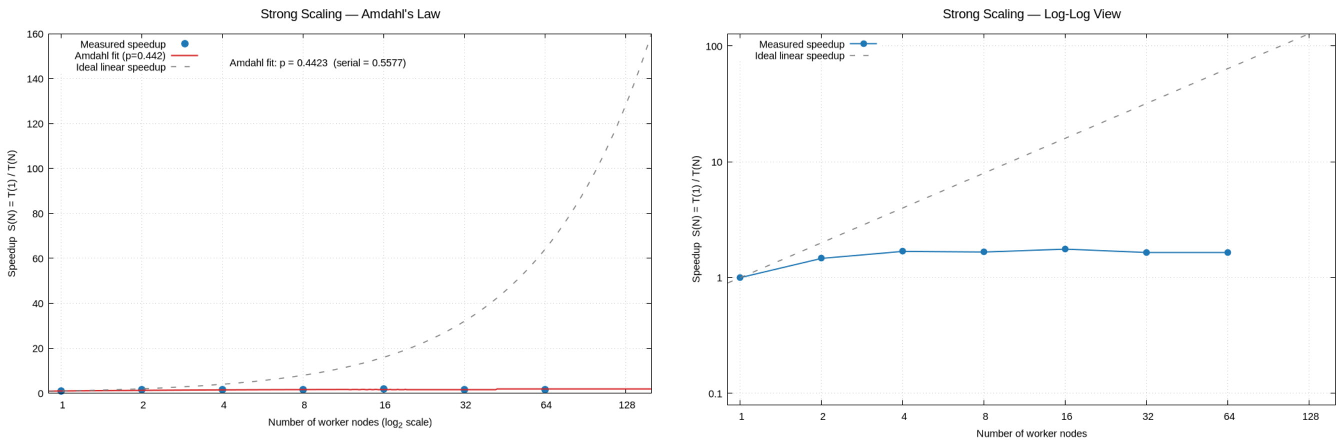

Figure 3: Measured speedup for the validated sweep. The left figure is the main Amdahl view with linear speedup on the y-axis; the right figure shows the same data on a log-log scale to emphasize the high-node-count collapse.

Could the graph change with multiple dataset sizes? Yes. Small datasets would saturate even earlier, while larger datasets would usually delay the point where fixed coordination costs become dominant. In this session, we tested a larger dataset (8 Common Crawl WET file chunks). The memory constraints caused crashes on the smaller node deployments (N=1, 2), but the scaling portion (N=4 to N=8) achieved a near-linear 1.84x speedup. This demonstrates that larger workloads per-node do help overcome the networking overhead compared to the tiny Go dataset limit. The generated plots for this secondary dataset are available as `amdahl_8.png` and `amdahl_8_loglog.png` in the report directory.

4.4 Batch x Stream Processing

In a direct comparison using the 64-file Go source dataset on 8 nodes, the custom Go MapReduce system completed the compute phase in **0.266 seconds**, whereas Kafka Streams took **10.095 seconds**. While Kafka's compute time is significantly higher on this small batch workload, this is an architectural trade-off: Kafka Streams persists every record to a replicated topic log (ensuring high durability and fault tolerance but paying a high per-record I/O cost). Conversely, our system is strictly batch-oriented and relies entirely on in-memory maps and local temporary disk buckets, which is extremely fast for small datasets but lacks data replication.

Furthermore, in a separate 50-node benchmark, Kafka Streams completed the wordcount job reliably in 248.5 seconds, whereas the custom Go MapReduce system struggled to deploy at that scale due to SSH connection limits and extreme memory overhead. Kafka manages dynamic partitions inherently, making it far more robust for continuous processing at scale.

5 Conclusion

The project successfully implements a coherent distributed MapReduce system in Go, together with a reusable deployment pipeline and a prepared Kafka comparison path. The main technical success is the detailed

MapReduce runtime: worker build injection, deployment, HTTP coordination, slot-preserving recovery, and pluggable jobs all work end to end.

For the report experiments, the validated maximum dataset processed was the fixed 64-file corpus used in the strong-scaling sweep. The measured results show that the system scales well only up to a modest number of nodes for wordcount. The best compute time was obtained at 8 nodes; after that, the reduce phase dominates and speedup collapses.

In short:

- the project works and is technically complete as a single-stage MapReduce system;
- the main remaining engineering work is scalability of the map/shuffle implementation, especially memory use and high-node-count reduce behavior;
- the cross-login validation still needs to be added before the report is fully final.